BrowserLens: Interactive Diagnosis of Web-Agent Trajectories with a Layered State-Action Graph

Junyeong Song
Department of Computer Science and Engineering
Ulsan National Institute of Science and Technology (UNIST)
Ulsan, Republic of Korea
magicwho@unist.ac.kr

Mingyu Kim
Department of Design
Ulsan National Institute of Science and Technology (UNIST)
Ulsan, Republic of Korea
gyu7991@unist.ac.kr

Seongkook Heo
Department of Computer Science and Engineering
Ulsan National Institute of Science and Technology (UNIST)
Ulsan, Republic of Korea
seongkook@unist.ac.kr

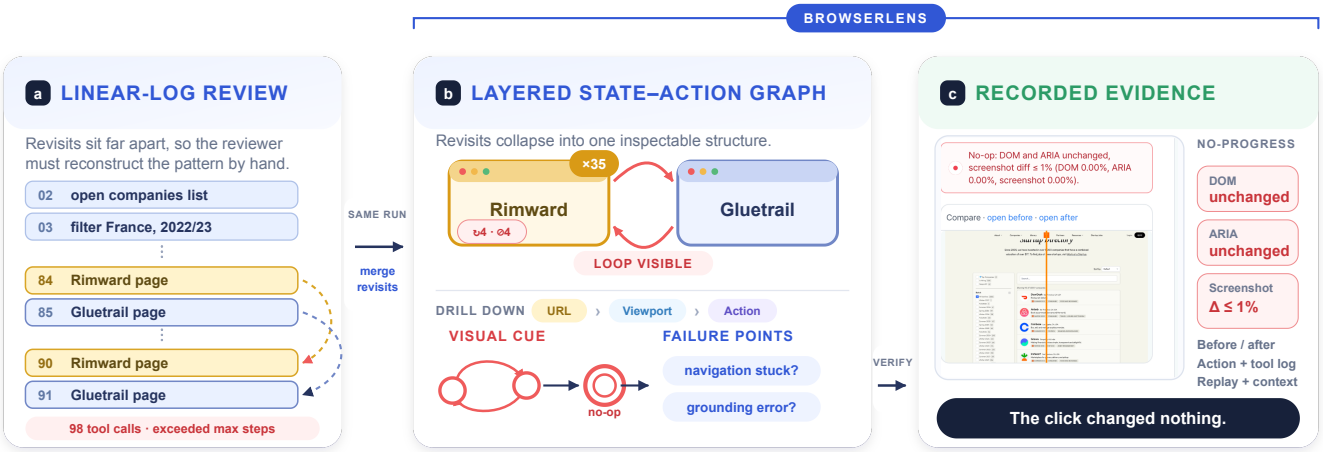


Figure 1: From scattered log entries to a verified diagnosis. (a) Revisits to the same pages sit far apart in the linear log; the run ends with *exceeded max steps*. (b) BrowserLens merges revisits by browser-state identity: the loop becomes a visible cycle, and drilling down reveals a ringed no-op cue. (c) Recorded evidence shows: DOM and ARIA unchanged, screenshots difference within 1%, meaning the click changed nothing.

Abstract

A completed web-agent run leaves only a trajectory: a long linear log in which cross-step structure such as revisits, loops, dead ends, and stalls is scattered across distant entries. We present BrowserLens, a read-only diagnosis workspace that reorganizes a single logged trajectory around browser-state identity rather than time. Normalized URLs, viewports, and state-anchored actions form a three-layer graph in which revisits collapse into single nodes and repetition becomes visible structure. Four visual cues mark suspicious structure; each cue is an attention signal, not a failure label, and links to the recorded evidence (screenshots, DOM/ARIA snapshots, action logs, LLM context, and replay) against which a reviewer confirms or rejects a failure hypothesis. We ground the design in benchmark-reported web-agent failures and in HCI work on event-sequence and browsing-history visualization.

This work is licensed under a Creative Commons Attribution 4.0 International License.

UIST Adjunct '26, Detroit, MI, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2855-6/2026/11
https://doi.org/10.1145/3830397.3841876

CCS Concepts

• **Human-centered computing** → **Visual analytics**; *Graphical user interfaces.*

Keywords

web agents, agent debugging, trajectory analysis, graph visualization, sensemaking

ACM Reference Format:
Junyeong Song, Mingyu Kim, and Seongkook Heo. 2026. BrowserLens: Interactive Diagnosis of Web-Agent Trajectories with a Layered State-Action Graph. In *The 39th Annual ACM Symposium on User Interface Software and Technology (UIST Adjunct '26), November 02–05, 2026, Detroit, MI, USA.* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3830397.3841876

1 Introduction

Web agents promise to carry out everyday web browsing tasks such as search, booking, shopping, and form filling from natural-language instructions [5]. Commercial computer-use systems [2, 8, 14] now act in live websites, and benchmarks increasingly evaluate them there [9, 21, 23]. However, these agents still remain far from reliable, and reported success rates often overstate actual progress [21], shifting the practical bottleneck to system diagnosis.

Developers, researchers, and operators now spend a growing share of their time reviewing completed runs. The question is rarely whether a run failed but where: the step that broke the goal, so a developer can repair the agent, a researcher can characterize the failure, and an operator can judge the outcome [6, 15]. What a run leaves behind, though, is only a trajectory log; nothing in it marks a natural unit to audit, so evaluation begins by reconstructing the agent's browser state and actions at each step.

Web-agent failures are difficult to diagnose because the relevant evidence is distributed across browser states and steps. Some failures are structural: loops over the same page, dead-end detours, or early mistakes that accumulate across the trajectory [9, 23, 24]. Others are local but state-dependent: a grounding error may occur in one step, yet deciding whether the click worked requires comparing the page before and after the action [12, 22]. A linear log preserves this evidence but scatters the structure needed to interpret it, turning diagnosis into a sensemaking task [16] that the reviewer must assemble by themselves.

Making an autonomous agent's behavior legible to the person who must judge it is a human-computer interaction (HCI) problem, and HCI offers relevant precedents for reorganizing temporal records into inspectable structure. Event-sequence visualizations reveal patterns that temporal listings hide [17, 20], and browsing-history tools organize human web activity by visited pages and screens to support revisitation and recall [1, 4, 10]. Yet current agent diagnosis tools still organize long runs around the agent's internal messages and tool calls rather than browser state [6, 7, 13, 15, 19]. A loop becomes visible only when distant visits are recognized as the same page, and a stalled action only when the page is seen not to change; ordered by the message stream, both remain buried. What is missing is a representation that organizes one web-agent trajectory by browser-state identity, surfaces suspicious structure, and lets reviewers verify what the structure suggests.

BrowserLens addresses this need with a read-only diagnosis workspace that reorganizes a logged trajectory around browser-state identity rather than chronological time. Each step is indexed by a deterministic identity extracted from the log, so revisits to the same state collapse into a single node and the run becomes a three-layer graph of pages, screens, and actions. In this graph, loops, detours, and stalls appear as visible structure. Because structure alone cannot settle a diagnosis, BrowserLens avoids automated failure labels: its visual cues direct attention, suggest benchmark-grounded failure points, and route the reviewer to recorded evidence that confirms or rejects each hypothesis.

2 Design Goals

We consider a reviewer examining a completed, logging-enabled run to determine whether and why it failed. BrowserLens is deliberately read-only; it does not diagnose hidden reasoning errors directly, but surfaces them when they appear as observable contradictions in browser state or recorded actions [21].

Existing interactive debuggers make recorded executions inspectable [3, 7, 11, 13, 19], but web-agent trajectories differ in three ways. *Unit:* browser state determines what an action means; the same click can be progress on the target page or a stall under an unclosed modal. Existing agent debuggers keep the message stream as their unit, whereas browsing research treats the page and screen as units of analysis [1, 10]. *Structure:* important failures unfold non-linearly; distant visits may close a loop, and a graph collapses them into one node with a visible cycle [18, 20]. *Interpretation:* structural signals are ambiguous and map onto causes many-to-many; a revisit may be a deliberate re-check or a trapped loop, so signals should guide review rather than declare failure.

These observations lead to three design goals. **DG1.** Make browser state the unit of diagnosis; merge instances that share an identity so revisits do not scatter. **DG2.** Expose non-linear trajectory structure, such as cycles, detours, and no-progress spans, with drill-down from overview to step. **DG3.** Treat signals as attention-directing cues; the reviewer confirms or rejects each hypothesis against recorded evidence.

3 BrowserLens

We implemented BrowserLens as a web app using a FastAPI backend and an HTML/JS frontend, powered by a Playwright runtime. The prototype parses each logged action step into four categories: page URLs, viewport geometry, executed actions, and before/after screenshots and DOM snapshots. Model context and replay artifacts are retained as supporting evidence rather than node identities.

BrowserLens builds a diagnosis index in which every step is keyed by deterministic browser-state identity, and every view in the workspace derives from that index. The workspace pairs timeline, graph, and stats views with a synchronized evidence panel. Review follows three stages (Figure 1): a visual cue draws attention, the cue suggests tentative failure points, and recorded evidence decides.

State identity (DG1). Identity is computed from logged state, never inferred by a model, so merging is reproducible. A URL identity keeps origin and path, drops fragments, strips tracking parameters, and sorts the remaining query, so parameters that carry task state, such as search terms, filters, and pagination, still split nodes. A viewport identity adds viewport size and scroll position within its URL. An action identity anchors the action signature, including tool and arguments, to the viewport it acted on, so one click on two screens stays two nodes.

Layered graph (DG2). The graph stack has three layers: URL, viewport, and state-anchored action. These layers match the questions reviewers ask: which page, which screen of that page, and what was done there. Revisits collapse into one node, with visit count encoded in node size and color. Double-clicking expands a node into the layer beneath.

Cues and evidence (DG3). Four cues annotate the graph (Figure 1b): repeated-node highlights, loop-edge highlights, no-op highlights, and hidden-cue badges. Hidden-cue badges summarize cues collapsed inside a parent node, so drill-down never buries a signal. A cue never asserts a failure; it marks structure consistent with tentative, benchmark-grounded failure points, such as repeated state or action, navigation loop, dead end, navigation stuck, or grounding error. The reviewer then settles each hypothesis against recorded evidence. The no-op cue fires only on explicit no-progress evidence: unchanged DOM and ARIA, screenshot difference of at most 1%. Selecting a node opens the deciding evidence: before/after screenshots, DOM/ARIA diff, action and tool log, replay, and model context (Figure 1c).

Walkthrough. Figure 1 shows a logged run that searched the Y Combinator directory for 2022/23 French startups with open jobs and ended with *exceeded max steps* after 98 tool calls; no single step looks broken. The URL layer reveals otherwise: one company page has accumulated 35 visits, its node enlarged and badged, with loop edges tracing the circuit between company pages. Drilling into the directory page's actions shows a repeated *All batches* click ringed as a no-op. Its evidence panel reports DOM and ARIA unchanged: the click changed nothing.

4 Future Directions

BrowserLens trades coverage for verifiability. Deterministic identity keeps the graph faithful to the log, but may split states when URLs contain volatile parameters or merge screens that are visually similar but semantically distinct. Likewise, BrowserLens currently surfaces structural failures that leave traces in browser-state, and supporting steering and repairing the behavior remain as future work. Our next step is a comparison with linear-log review on failure-rich trajectories, focusing on how reviewers notice, interpret, and explain failures.

Acknowledgments

This work was supported by the InnoCORE program of the Ministry of Science and ICT (UNIST 2.260039.01).

References

[1] Eytan Adar, Jaime Teevan, and Susan T. Dumais. 2008. Large Scale Analysis of Web Revisitation Patterns. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 1197–1206.

[2] Anthropic. 2024. Introducing Computer Use, a New Claude 3.5 Sonnet, and Claude 3.5 Haiku. https://www.anthropic.com/news/3-5-models-and-computer-use.

[3] Brian Burg, Richard Bailey, Amy J. Ko, and Michael D. Ernst. 2013. Interactive Record/Replay for Web Application Debugging. In *Proceedings of the 26th Annual ACM Symposium on User Interface Software and Technology*. 473–484.

[4] Daniel Cernea, Igor Truderung, Andreas Kerren, and Achim Ebert. 2013. WebComets: A Tab-Oriented Approach for Browser History Visualization. In *GRAPP/IVAPP*. 439–450.

[5] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems*.

[6] Michael Desmond, Jae Young Lee, Ibrahim Ibrahim, James M. Johnson, Avirup Sil, James MacNair, and Ruchir Puri. 2025. Agent Trajectory Explorer: Visualizing and Providing Feedback on Agent Trajectories. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 29634–29636.

[7] Will Epperson, Gagan Bansal, Victor C. Dibia, Adam Fourney, Jack Gerrits, Erkang Zhu, and Saleema Amershi. 2025. Interactive Debugging and Steering of Multi-Agent AI Systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*.

[8] Google DeepMind. 2025. Introducing the Gemini 2.5 Computer Use Model. https://blog.google/technology/google-deepmind/gemini-computer-use-model/.

[9] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 6864–6890.

[10] Donghan Hu and Sang Won Lee. 2020. ScreenTrack: Using a Visual History of a Computer Screen to Retrieve Documents and Web Pages. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. 1–13.

[11] Amy J. Ko and Brad A. Myers. 2004. Designing the Whyline: A Debugging Interface for Asking Questions About Program Behavior. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 151–158.

[12] Eric Li and Jim Waldo. 2024. WebSuite: Systematically Evaluating Why Web Agents Fail. *arXiv preprint arXiv:2406.01623* (2024).

[13] Jieyu Lu, Bo Pan, Jieyi Chen, Yingchaojie Feng, Jingyu Hu, Yuchen Peng, and Wei Chen. 2024. AgentLens: Visual Analysis for Agent Behaviors in LLM-Based Autonomous Systems. *IEEE Transactions on Visualization and Computer Graphics* (2024).

[14] OpenAI. 2025. Introducing Operator. https://openai.com/index/introducing-operator/.

[15] Tianyue Ou, Wei Guo, Anuj Gandhi, Graham Neubig, and Xiang Yue. 2025. AgentDiagnose: An Open Toolkit for Diagnosing LLM Agent Trajectories. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 207–215.

[16] Peter Pirolli and Stuart Card. 2005. The Sensemaking Process and Leverage Points for Analyst Technology as Identified Through Cognitive Task Analysis. In *Proceedings of the International Conference on Intelligence Analysis*, Vol. 5. 2–4.

[17] Catherine Plaisant, Brett Milash, Anne Rose, Seth Widoff, and Ben Shneiderman. 1996. LifeLines: Visualizing Personal Histories. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 221–227.

[18] Yaoyao Qian, Yuanli Wang, Jinda Zhang, Yun Zong, Mingzhe Chen, Haonan Zhou, et al. 2025. WebGraphEval: Multi-Turn Trajectory Evaluation for Web Agents Using Graph Representation. *arXiv preprint arXiv:2510.19205* (2025).

[19] Renqi Sheng, Yifei Yang, Chuhan Shi, Yifan Lin, Zhuo Chen, Huamin Qu, and Furui Cheng. 2026. DiLLS: Interactive Diagnosis of LLM-Based Multi-Agent Systems via Layered Summary of Agent Behaviors. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*.

[20] Krist Wongsuphasawat, John Alexis Guerra Gómez, Catherine Plaisant, Taowei David Wang, Meirav Taieb-Maimon, and Ben Shneiderman. 2011. LifeFlow: Visualizing an Overview of Event Sequences. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 1747–1756.

[21] Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. 2025. An Illusion of Progress? Assessing the Current State of Web Agents. *arXiv preprint arXiv:2504.01382* (2025).

[22] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. GPT-4V(ision) Is a Generalist Web Agent, If Grounded. *arXiv preprint arXiv:2401.01614* (2024).

[23] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*.

[24] Kexun Zhu, Zhenyu Liu, Bocheng Li, Muyang Tian, Yusen Yang, Jiaqi Zhang, et al. 2025. Where LLM Agents Fail and How They Can Learn from Failures. *arXiv preprint arXiv:2509.25370* (2025).